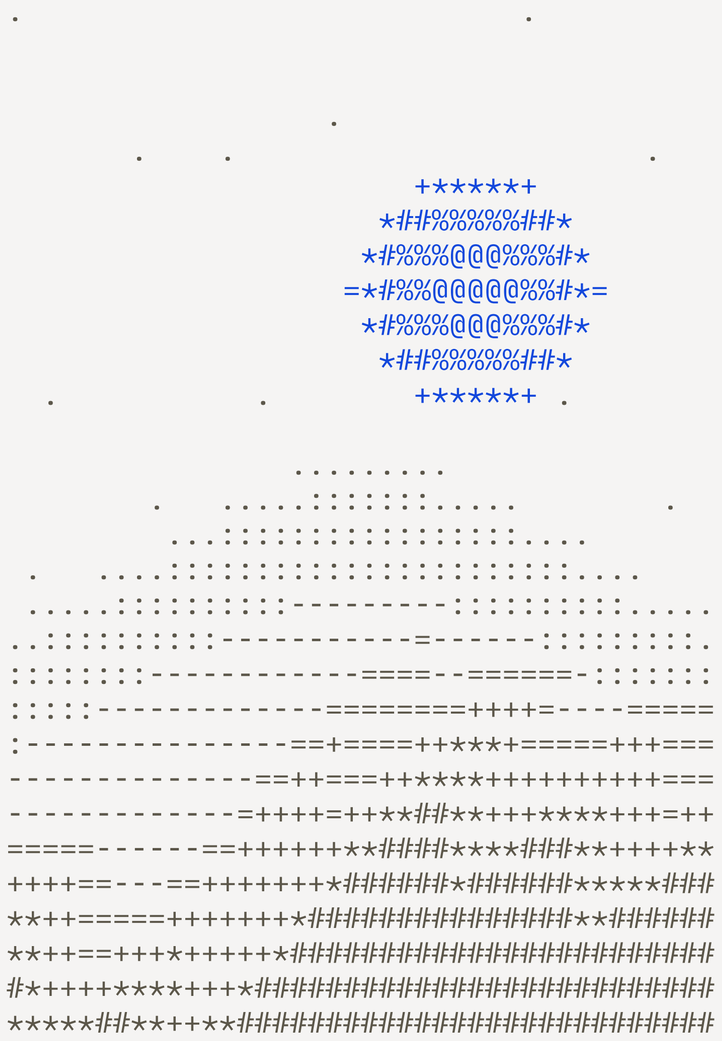
[PROGRESS REPORT · NO.07]

Sparse Attention for Long-Document OCR

长文档 OCR 中的稀疏注意力 — weekly update on training & eval



presenter	Your Name
advisor	Prof. Advisor
lab	ABC institution
period	2026.10.09 → 2026.10.16

[00] AGENDA

This week at a glance

[01] TL;DR & status p.03

[02] **Experiments: sparse vs. dense** p.04

[03] Results on DocVQA-long p.06

[04] Blockers & questions p.08

[05] Plan for next week p.09

[01] TL;DR

Sparse attention matches dense accuracy at 3.1× less memory

[x]	Re-implemented block-sparse mask in FlashAttention-2 kernel	done · 10.12
[x]	Full DocVQA-long eval for 4k / 8k / 16k context	done · 10.14
[~]	Ablation: global-token count (8 → 64)	3 / 5 runs
[!]	OOM on 32k context with batch size > 1	blocked
[ ]	Write related-work section for the workshop draft	next week

02

[SECTION 02 / 05]

Experiments

Sparse vs. dense attention on long scanned documents — setup, runs, and what broke.

[02] METHOD

Block-sparse attention keeps local windows + a few global tokens

- Each 64-token block attends to itself and its 2 neighbours
- 16 global tokens (layout anchors) see every block
- Mask is built once per page, reused across layers
- Drop-in for the dense ViT encoder — no retraining of the decoder

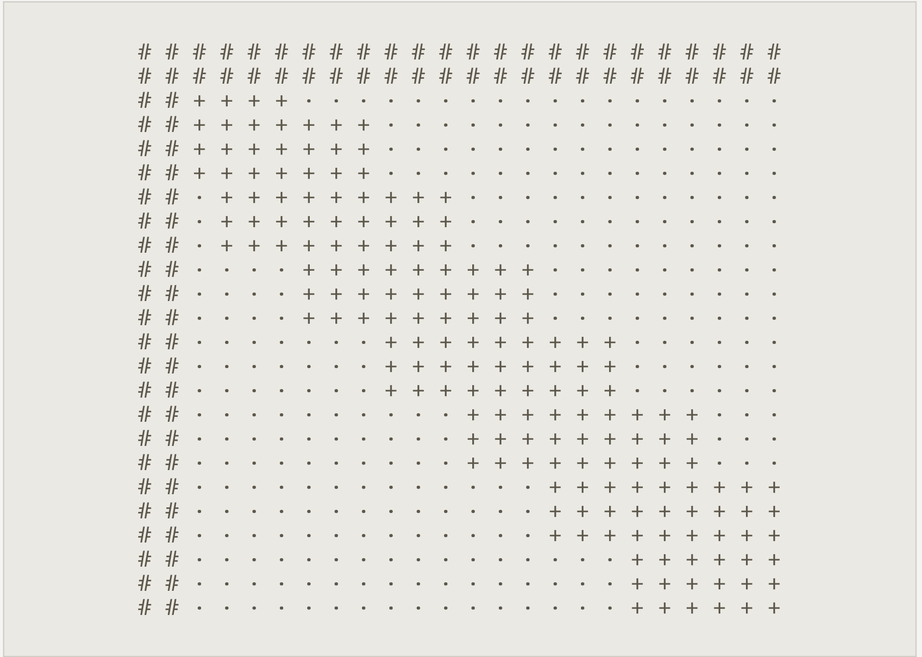


fig.01 — attention mask (# global, + local, · skipped)

[03] RESULTS

Same accuracy, a third of the memory

+0.4 ↑
ANLS vs. dense

3.1× ↓
less GPU memory @16k

1.8× ↑
faster inference

MODEL	CTX	ANLS	MEM (GB)	TOK/S
Dense ViT-L (baseline)	16k	71.2	38.4	412
Longformer-style	16k	69.8	15.1	690
Block-sparse (ours)	**16k**	**71.6**	**12.3**	**745**

[04] IMPLEMENTATION

The whole mask is ten lines of PyTorch

sparse_mask.py | train.py

```python
def build_mask(n_blocks, window=1, n_global=16):
    """Block-sparse mask: local window + global."""
    idx = torch.arange(n_blocks)
    dist = (idx[:, None] - idx[None, :]).abs()
    local = dist <= window

    glob = torch.zeros_like(local)
    glob[:n_global, :] = True
    glob[:, :n_global] = True
    return local | glob
```

→ L4–5

band of width 1 → 3 neighbouring blocks

→ L7–9

global rows + cols; anchors are the first 16 layout tokens

→ L10

boolean OR, fed straight to FlashAttention's block mask

[05] BLOCKERS & QUESTIONS

Two things I'd like your input on

[!] BLOCKER

32k context OOMs at batch > 1

Activation memory of the decoder dominates now. Gradient checkpointing helps but costs 35% throughput.

[?] QUESTION

Which venue should we target?

Results are workshop-ready. Is ICDAR'27 (deadline 01.15) realistic, or wait for the full ablation?

[i] NOTE

Cluster quota resets on 10.20

Queued 3 ablation runs to start Monday; ETA Wednesday.

→ decision needed target venue, by Thu 10.23 — so I can plan the ablation budget

[06] NEXT WEEK

Plan for W43: finish ablations, start writing

TODO (2)

- Related-work section
  writing · 2d
- Rerun baseline w/ new tokenizer
  exp · 4h

DOING (2)

- Global-token ablation (8→64)
  3 / 5 runs
- Fix 32k OOM via chunked decoder
  debug

DONE (2)

- Sparse kernel + unit tests
  merged #41
- DocVQA-long eval @ 4k/8k/16k
  logged

[EOF]

Thanks.

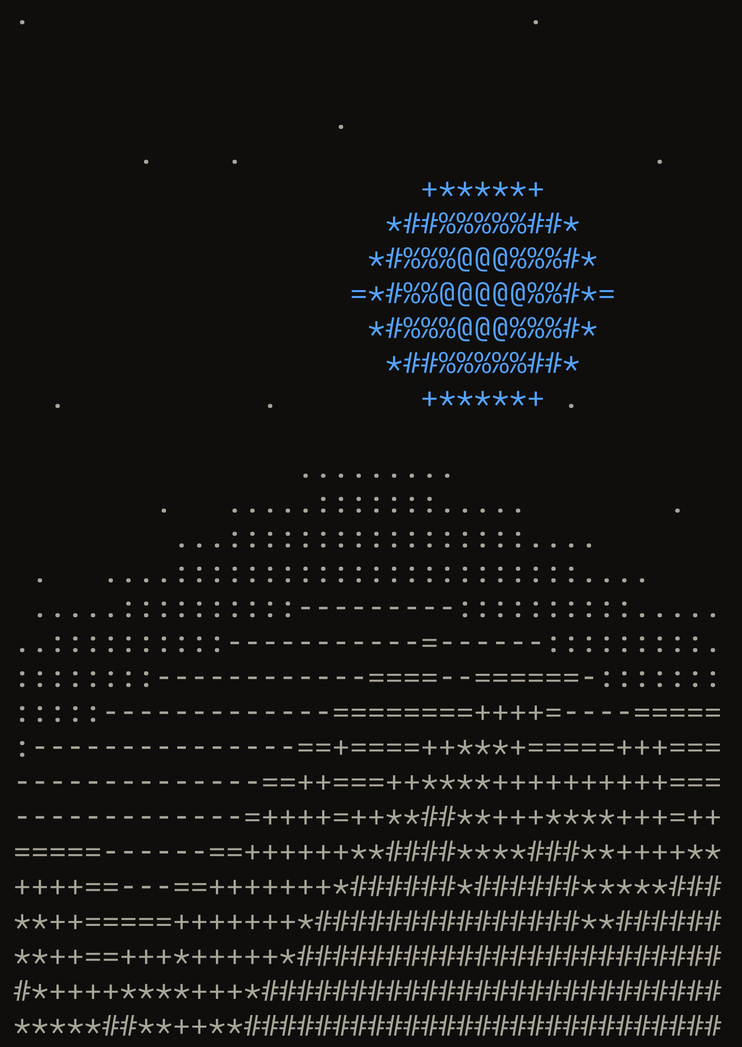
questions · comments · suggestions → Q&A



[PROGRESS REPORT · NO.07]

Sparse Attention for Long-Document OCR

长文档 OCR 中的稀疏注意力 — weekly update on training & eval



presenter	Your Name
advisor	Prof. Advisor
lab	ABC institution
period	2026.10.09 → 2026.10.16

[02] METHOD

Block-sparse attention keeps local windows + a few global tokens

- Each 64-token block attends to itself and its 2 neighbours
- 16 global tokens (layout anchors) see every block
- Mask is built once per page, reused across layers
- Drop-in for the dense ViT encoder — no retraining of the decoder

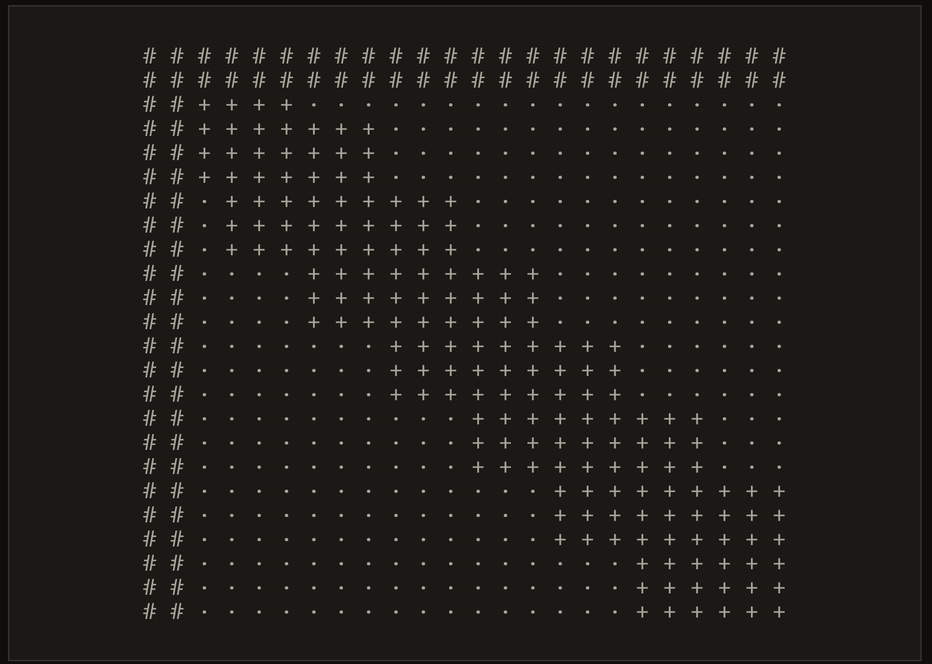


fig.01 — attention mask (# global, + local, · skipped)

[03] RESULTS

Same accuracy, a third of the memory

+0.4 ↑
ANLS vs. dense

3.1× ↓
less GPU memory @16k

1.8× ↑
faster inference

MODEL	CTX	ANLS	MEM (GB)	TOK/S
Dense ViT-L (baseline)	16k	71.2	38.4	412
Longformer-style	16k	69.8	15.1	690
Block-sparse (ours)	**16k**	**71.6**	**12.3**	**745**

[04] IMPLEMENTATION

The whole mask is ten lines of PyTorch

sparse_mask.py | train.py

```python
def build_mask(n_blocks, window=1, n_global=16):
    """Block-sparse mask: local window + global."""
    idx = torch.arange(n_blocks)
    dist = (idx[:, None] - idx[None, :]).abs()
    local = dist <= window

    glob = torch.zeros_like(local)
    glob[:n_global, :] = True
    glob[:, :n_global] = True
    return local | glob
```

→ L4–5
band of width 1 → 3 neighbouring blocks

→ L7–9
global rows + cols; anchors are the first 16 layout tokens

→ L10
boolean OR, fed straight to FlashAttention's block mask

[EOF]

Thanks.

questions · comments · suggestions → Q&A

